

Computing response time distributions using stochastic Petri nets and matrix diagrams

Andrew S. Miner

Dept. of Computer Science,
Iowa State University, Ames, Iowa, 50011
E-mail: asminer@iastate.edu

Abstract

In this paper, we consider random variables expressed in terms of the time required for the state of a stochastic Petri net to pass from a set of starting markings to a set of stopping markings. These random variables have continuous phase-type distributions when the all transitions have exponentially-distributed firing delays. We demonstrate how to numerically compute the distribution of the random variable using both explicit techniques and an implicit approach based on multi-way decision diagrams and matrix diagrams. We present an efficient matrix-vector multiplication algorithm for matrix diagrams that is necessary for numerical solution. We demonstrate the efficiency of our approaches using several models. The lower storage requirements of the implicit approach effectively increases the size of models that can be analyzed by about an order of magnitude.

1. Introduction

Stochastic Petri nets (SPNs) are widely accepted as a useful tool for performance evaluation. By generating and analyzing the underlying stochastic process of the SPN, usually a continuous-time Markov chain (CTMC), researchers can obtain useful performance measures about the system. This type of analysis usually involves the determination of a transient or stationary probability distribution for the CTMC. Such well-established techniques are able to provide *expected values* for various quantities, such as queue length, time until failure, or response time. However, since a user's perception of a system is based upon the response times of *individual* requests, rather than the *mean* response time, it is important to study the distribution of the response time in such systems. This paper concerns the efficient numerical computation of distributions of random variables expressed in terms of a stochastic Petri net and the

passage time from an initial set of markings to a final set of markings.

Traditionally, response time distributions have been studied numerically for various classes of queuing networks, for example [22, 24]. For more general cases, discrete-event simulation can be used, although this can be computationally expensive. Another general approach, based on analysis of differential equations via Laplace transform inversion, has been explored recently [15, 18]. Earlier work [28] used stochastic reward nets to obtain response time distributions $\Pr\{R \leq t\}$ via transient analysis. We improve upon [28] by using implicit techniques (which do not require explicit storage of the underlying Markov chain) to combat the state explosion problem, and customize our approach to the computation of distributions.

There is a large body of related work based on implicit techniques; for example, multi-terminal binary decision diagrams (MTBDDs) can be used to represent and analyze large CTMCs [20]. The use of uniformization to perform model checking of CTMCs (e.g., [4]) is quite similar to our work; the primary difference is that [4] performs uniformization for a single time t , while we are concerned with a large, finite set of times. Kronecker representations have also been explored for both stationary [5] and transient analysis [23]. Matrix diagrams [11] have been explored as an alternative to Kronecker representations for stationary analysis, and in this paper we show how they can be applied to determine response time distributions. The technique we describe could be applied also to Kronecker representations or to MTBDDs, and could be incorporated into model checking tools.

The remainder of the paper is organized as follows. Section 2 gives an overview of phase-type distributions as they relate to our work. Section 3 describes our modeling formalism and how response time distributions can be computed explicitly. Section 4 gives a brief overview of the implicit data structures we use. Section 5 describes how the implicit data structures can be used to compute the response time distributions. Experimental results are given in

Section 6. Finally, Section 7 concludes the paper and gives some directions for future work.

2. Phase-type distributions

We introduce the notation used in the paper and briefly review the well-known properties of phase-type distributions [29] that are relevant for this work. We denote the set of naturals as $\mathbb{N}$, the set of reals as $\mathbb{R}$, and the set of non-negative reals as $\mathbb{R}^+$; all other sets are denoted in script: $\mathcal{I}$. Row vectors and matrices are denoted in lower-case and upper-case boldface: $\mathbf{x}$, $\mathbf{A}$. The identity matrix is denoted $\mathbf{I}$, and $\mathbf{1}^\top$ denotes a column vector of ones. Individual vector and matrix elements are denoted using subscripts, e.g., x_i , A_{ij} ; sets are used to indicate subvectors and submatrices: $\mathbf{x}_{\mathcal{I}}$, $\mathbf{A}_{\mathcal{I}\mathcal{J}}$.

2.1. Discrete phase-types

Discrete phase-type random variables are those that can be described as the time until absorption of a discrete-time Markov chain (DTMC). Formally, a random variable X is a discrete phase-type if there exists an absorbing DTMC with transition probability matrix $\mathbf{P}$, absorbing states $\mathcal{A}$, transient states $\mathcal{Z}$ (with $\mathcal{A} \cap \mathcal{Z} = \emptyset$), and an initial probability distribution $\boldsymbol{\pi}$ such that X is equal to the time until the DTMC is absorbed into one of the states in $\mathcal{A}$. The probability mass function (PMF) for X can be expressed in terms of the matrix $\mathbf{P}$:

$$\Pr\{X = n\} = \begin{cases} \boldsymbol{\pi}_{\mathcal{Z}} \mathbf{P}_{\mathcal{Z}\mathcal{Z}}^{n-1} \mathbf{P}_{\mathcal{Z}\mathcal{A}} \mathbf{1}^\top & \text{if } n > 0 \\ \boldsymbol{\pi}_{\mathcal{A}} \mathbf{1}^\top & \text{if } n = 0 \end{cases} \quad (1)$$

Similarly, it can be seen that $\Pr\{X > n\} = \boldsymbol{\pi}_{\mathcal{Z}} \cdot \mathbf{P}_{\mathcal{Z}\mathcal{Z}}^n \cdot \mathbf{1}^\top$, by noticing that X is larger than n if and only if the DTMC is still in a transient state at time n . This can be used in practice to truncate the computation of the PMF of X : we select a desired precision ϵ , and compute the PMF for values of n until $\Pr\{X > n\} < \epsilon$.

2.2. Continuous phase-types

A continuous phase-type random variable Z can be expressed as the time until absorption of a continuous-time Markov chain (CTMC) with infinitesimal generator matrix $\mathbf{Q}$, absorbing states $\mathcal{A}$, transient states $\mathcal{Z}$, and initial probability distribution $\boldsymbol{\pi}$. The probability density function (PDF) for Z can be written in terms of the matrix exponential: $f(t) = \boldsymbol{\pi}_{\mathcal{Z}} \cdot e^{\mathbf{Q}_{\mathcal{Z}\mathcal{Z}} \cdot t} \cdot \mathbf{Q}_{\mathcal{Z}\mathcal{A}} \cdot \mathbf{1}^\top$. Using the technique of uniformization [17], the CTMC can be “converted” into a DTMC, by selecting $q \geq |\mathbf{Q}_{ii}|$ for all diagonal elements of $\mathbf{Q}$ and constructing the probability matrix $\mathbf{P} = \frac{1}{q} \cdot \mathbf{Q} + \mathbf{I}$.

We can express $f(t)$ in terms of the obtained DTMC:

$$f(t) = \boldsymbol{\pi}_{\mathcal{Z}} \sum_{n=0}^{\infty} \frac{t^n}{n!} \cdot \mathbf{Q}_{\mathcal{Z}\mathcal{Z}}^n \mathbf{Q}_{\mathcal{Z}\mathcal{A}} \mathbf{1}^\top \quad (2)$$

$$= q \sum_{n=0}^{\infty} \Pr\{Y(qt) = n\} \cdot \Pr\{X = n+1\} \quad (3)$$

where X is the discrete phase-type random variable with matrix $\mathbf{P}$ and initial probability distribution $\boldsymbol{\pi}$, and $Y(qt)$ is a Poisson random variable with parameter qt .

The cumulative distribution function (CDF) can also be written using the matrix exponential, as $F(t) = \Pr\{Z \leq t\} = 1 - \Pr\{Z > t\} = 1 - \boldsymbol{\pi}_{\mathcal{Z}} \cdot e^{\mathbf{Q}_{\mathcal{Z}\mathcal{Z}} \cdot t} \cdot \mathbf{1}^\top$. Using uniformization again, we can obtain

$$\Pr\{Z > t\} = \sum_{n=0}^{\infty} \Pr\{Y(qt) = n\} \cdot \Pr\{X > n\} \quad (4)$$

where X and $Y(qt)$ are defined as above. Equation (3) illustrates an efficient way to compute $f(t)$ for a set of values of t : we first compute the PMF of X based on (1), which requires a series of vector-matrix multiplications. We can then obtain $f(t)$ for as many values of t as we like by computing the distribution of $Y(qt)$, which can be done efficiently using the algorithm of Fox and Glynn [16], and then computing the sum shown in (3). We can compute $F(t)$ in a similar way, using (4) instead.

3. First passage times of SPNs

We specify systems using a fairly general class of stochastic Petri nets, including inhibitor arcs, transition guards, and marking-dependent arc cardinalities. We assume that all firing times are exponentially distributed (possibly with marking-dependent rates), so that the underlying process is a CTMC [27]. Thus, we consider stochastic Petri nets of the form $(\mathcal{P}, \mathcal{T}, f, h, g, \lambda, \mathcal{M}_0, \boldsymbol{\pi})$, where

- $\mathcal{P}$ is the set of places. A marking $\mathbf{m} \in \mathbb{N}^{|\mathcal{P}|}$ specifies a number of tokens for each place.
- $\mathcal{T}$ is the set of transitions, with $\mathcal{P} \cap \mathcal{T} = \emptyset$.
- $f : (\mathcal{P} \times \mathcal{T} \cup \mathcal{T} \times \mathcal{P}) \times \mathbb{N}^{|\mathcal{P}|} \rightarrow \mathbb{N}$ specifies the input and output arc cardinality functions.
- $h : \mathcal{P} \times \mathcal{T} \times \mathbb{N}^{|\mathcal{P}|} \rightarrow \mathbb{N} \cup \{\infty\}$ specifies the inhibitor arc cardinality functions.
- $g : \mathcal{T} \times \mathbb{N}^{|\mathcal{P}|} \rightarrow \{0, 1\}$ specifies the transition guards. A transition $t \in \mathcal{T}$ is *enabled* in a marking $\mathbf{i}$ if $g(\mathbf{i}) = 1$ and, for all places $p \in \mathcal{P}$, $f_{p,t}(\mathbf{i}) \leq \mathbf{i}_p < h_{p,t}(\mathbf{i})$; if enabled, the transition may *fire*, leading to marking $\mathbf{j}$ with $\mathbf{j}_p = \mathbf{i}_p - f_{p,t}(\mathbf{i}) + f_{t,p}(\mathbf{i})$ for all places $p \in \mathcal{P}$. We write $\mathbf{i} \xrightarrow{t} \mathbf{j}$ if t is enabled in marking $\mathbf{i}$, and the firing of t leads to marking $\mathbf{j}$.

- $\lambda : \mathcal{T} \times \mathcal{N}^{|\mathcal{P}|} \rightarrow \mathbb{R}^*$ specifies the marking-dependent firing rate for each transition: if transition $t \in \mathcal{T}$ is enabled in marking $\mathbf{i}$, then its firing time is exponentially distributed with rate $\lambda_t(\mathbf{i})$.
- $\mathcal{M}_0 \subset \mathcal{N}^{|\mathcal{P}|}$ is the non-empty set of initial markings.
- $\pi \in \mathbb{R}^{|\mathcal{M}_0|}$ is a probability vector, specifying the initial probabilities for the initial markings.

We say marking $\mathbf{i}$ is one-step reachable from marking $\mathbf{j}$, written $\mathbf{i} \rightarrow \mathbf{j}$, if $\mathbf{i} \xrightarrow{t} \mathbf{j}$ for some transition $t \in \mathcal{T}$. The reachability set of a Petri net is defined as the smallest set $\mathcal{S} \subseteq \mathcal{N}^{|\mathcal{P}|}$ such that $\mathcal{M}_0 \subseteq \mathcal{S}$ and $\mathcal{S}$ is closed under one-step reachability: if $\mathbf{i} \in \mathcal{S}$ and $\mathbf{i} \rightarrow \mathbf{j}$, then $\mathbf{j} \in \mathcal{S}$. For the remainder of the paper, we assume that $\mathcal{S}$ is finite. We define the transition rate matrix due to transition t , written $\mathbf{R}^t$, as a square matrix of size $|\mathcal{S}|$, where $\mathbf{R}_{ij}^t = \lambda_t(\mathbf{i})$ if $\mathbf{i} \xrightarrow{t} \mathbf{j}$; otherwise, $\mathbf{R}_{ij}^t = 0$. The underlying stochastic process of the Petri net is a CTMC with transition rate matrix $\mathbf{R} = \sum_{t \in \mathcal{T}} \mathbf{R}^t$. The infinitesimal generator matrix $\mathbf{Q}$ is equal to $\mathbf{R}$ except for the diagonal elements, which are defined so that each row of $\mathbf{Q}$ sums to zero.

To allow a general description of the desired response time distribution, we specify a set of starting markings $\mathcal{I} \subseteq \mathcal{S}$, an initial distribution π' for the starting markings, and stopping markings $\mathcal{J} \subset \mathcal{N}^{|\mathcal{P}|}$. Let the random variable Z be the time instant when the Petri net first reaches one of the stopping markings $\mathcal{J}$, given that the Petri net is in marking $\mathbf{i} \in \mathcal{I}$ at time 0 with probability π'_i . The problem we consider in this paper is how to efficiently compute the PDF or CDF of the random variable Z . For simplicity, we assume that $\mathcal{I} \cap \mathcal{J} = \emptyset$; otherwise, we must handle the possibility that $\Pr\{Z = 0\} > 0$.

It can be seen that Z is a continuous phase-type random variable. Let $\mathcal{S}'$ be the smallest set such that $\mathcal{I} \subseteq \mathcal{S}'$ and $\mathcal{S}'$ is closed under one-step reachability except for stopping markings: if $\mathbf{i} \in \mathcal{S}' \setminus \mathcal{J}$ and $\mathbf{i} \rightarrow \mathbf{j}$, then $\mathbf{j} \in \mathcal{S}'$. Note that, since $\mathcal{I} \subseteq \mathcal{S}$, we know that $\mathcal{S}' \subseteq \mathcal{S}$. Random variable Z is equal to the time until absorption of the CTMC with transient states $\mathcal{Z} = \mathcal{S}' \setminus \mathcal{J}$, absorbing states $\mathcal{A} = \mathcal{S}' \cap \mathcal{J}$, generator matrix $\mathbf{Q}'$ where $\mathbf{Q}'_{\mathcal{Z}\mathcal{Z}} = \mathbf{Q}_{\mathcal{Z}\mathcal{Z}}$ and $\mathbf{Q}'_{\mathcal{Z}\mathcal{A}} = \mathbf{Q}_{\mathcal{Z}\mathcal{A}}$, and initial probability distribution according to π' .

3.1. Running example

As a simple example, we use the readers and writers Petri net shown in Figure 1(a), with $N = 1$. Places p_1 and p_7 represent a pool of writer processes and reader processes, respectively; places p_2 and p_6 represent writers and readers waiting, places p_3 and p_5 represent processes writing and reading, and place p_4 is used for synchronization of the resource. Note that multiple reader processes can read simultaneously, but a writer process requires exclusive access

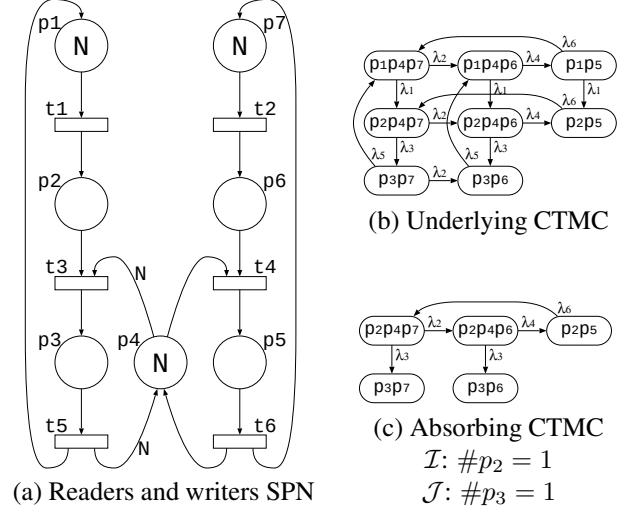


Figure 1. Running example

to the resource. Each transition t_i has an exponentially distributed firing delay with rate λ_i . The underlying CTMC for this net (with $N = 1$) is shown in Figure 1(b). To determine the waiting time of a process that wishes to perform a write operation, we use the markings with a token in place p_2 for the starting set $\mathcal{I}$, and we use the markings with a token in place p_3 for the stopping set $\mathcal{J}$. The absorbing CTMC for the phase-type variable Z is shown in Figure 1(c). Note that, to determine the PDF or CDF of Z , we must specify the initial distribution π' for the three initial markings in $\mathcal{I}$.

3.2. Determining initial probabilities

Before computing the distribution of Z , we must compute the initial probability distribution π' . If there is exactly one reachable marking in the set $\mathcal{I}$, then this is trivial: π' is the scalar 1. However, frequently it is the case that $\mathcal{I}$ contains several reachable markings; in these situations, we must determine π' . Different initial distributions π' may be meaningful, for instance:

- π' could be computed according to the probability that $\mathbf{i}$ is the first marking reached of those in the set $\mathcal{I}$, given that the net starts in one of the initial markings $\mathcal{M}_0$ with initial distribution π .
- π' could be computed according to the relative probability that the net enters marking $\mathbf{i}$ (given that it enters one of the markings in $\mathcal{I}$) due to a transition firing in steady-state. This would be computed by analyzing the embedded DTMC of the net.
- π' could be computed as above, but only after certain transitions fire, or after a given time.

Additionally, for certain specific problems, there may be efficient techniques to obtain the initial distribution; for example, using the “tagged customer” approach for product-form queuing networks, the initial probabilities can be computed by analyzing the system with one fewer customers [28, 30].

3.3. Explicit solution

Construction of the necessary structures to compute the PDF or CDF of Z can be done explicitly in a straightforward manner. The reachability set $\mathcal{S}$ for the original net can be generated using an exhaustive search and an appropriate data structure to store the markings [6, 10, 19]. The sets $\mathcal{I}$ and $\mathcal{J}$ can be specified as boolean functions, and the state space $\mathcal{S}'$ and the absorbing CTMC $\mathbf{Q}'$ can be constructed via breadth-first search. Once the underlying CTMC $\mathbf{Q}'$ has been constructed (and the initial probabilities π' have been computed or specified), the PDF can be computed using (3), and the CDF can be computed using (4).

4. Implicit representations

The benefits of implicit (also known as “symbolic”) approaches, such as those based on binary decision diagrams (BDDs) and their variants, are that the computation time and storage space required are not necessarily proportional to the number of reachable markings. For many practical models, implicit techniques are able to cope with much larger reachability sets than explicit techniques.

To use the implicit representations described in this section, the model must be composed of K submodels; for Petri nets, this is achieved by partitioning the set of places $\mathcal{P}$ into $\mathcal{P}_K, \dots, \mathcal{P}_1$ (we discuss the restrictions for doing so in Section 5). Thus, a marking can be expressed as a vector of K submarkings, where submarking k corresponds to the places in $\mathcal{P}_k$. We denote the set of reachable submarkings for subnet k as $\mathcal{S}_k$. While previous work assumed that the reachable submarkings can be generated in isolation *a priori*, recent work [9] has removed this restriction. The set of *potential* markings is defined as $\hat{\mathcal{S}} = \mathcal{S}_K \times \dots \times \mathcal{S}_1$. Note that the reachability set $\mathcal{S}$ is a subset of $\hat{\mathcal{S}}$; it is a strict subset if some combinations of submarkings cannot be reached.

Continuing with our running example, we can partition the set of places into $\mathcal{P}_6 = \{p_7\}$, $\mathcal{P}_5 = \{p_6\}$, $\mathcal{P}_4 = \{p_5, p_4\}$, $\mathcal{P}_3 = \{p_3\}$, $\mathcal{P}_2 = \{p_2\}$, and $\mathcal{P}_1 = \{p_1\}$. For the case $N = 1$, we have sets of reachable submarkings $\mathcal{S}_6 = \mathcal{S}_5 = \mathcal{S}_3 = \mathcal{S}_2 = \mathcal{S}_1 = \{0, 1\}$ and $\mathcal{S}_4 = \{(p_4), (p_5), (00)\}$.

4.1. Multivalued decision diagrams

To represent sets of markings and their indices, we use edge-valued, multi-way decision diagrams (EV⁺MDDs) [12], a data structure based on multi-way decision diagrams

(MDDs) [21]. Formally, an EV⁺MDD is a directed, acyclic graph used to represent functions of the form $f : \hat{\mathcal{S}} \rightarrow \mathbb{N} \cup \{\infty\}$. The graph contains a terminal node, labeled with zero, and non-terminal nodes, each labeled with a variable x_k . We say that a non-terminal node with label x_k is a level- k node, and that a terminal node is a level-0 node. A level- k non-terminal node contains $|\mathcal{S}_k|$ pointers to other nodes, which must be at a level less than k . Each pointer has an associated edge value, which is either infinity or a natural. The value of the function for particular variable assignments is found by traversing the graph (by following the appropriate pointers) and summing the edge values. Rules for reducing EV⁺MDDs and a proof that reduced EV⁺MDDs are a canonical representation can be found in [12].

We use a K -level EV⁺MDD to represent an indexing function for the potential markings: unreachable markings are assigned an index value of infinity, and reachable markings are assigned a unique index between 0 and $|\mathcal{S}| - 1$. For example, the reachability set for our running example can be represented using the EV⁺MDD shown in Figure 2(a). For clarity, pointers associated with values of infinity (which must be directly to terminal node 0) are omitted from the figure. Also, we stress that the submarkings are not stored in the EV⁺MDD itself; instead, the submarkings of $\mathcal{S}_k$ are each assigned a unique integer index, and arrays of pointers are used for each node, where the submarking index determines the position of the downward pointer. In the example, we find that all markings of the form $(\bullet p_6 p_7)$ are unreachable: following the pointer in the root node for $(\#p_7 = 1)$, and then following the pointer for $(\#p_6 = 1)$, leads to terminal node 0 with an edge-value sum of infinity. Conversely, marking $(p_3 p_7)$ is reachable, because the corresponding path has an edge-value sum of $5 + 0 + 2 + 0 + 0 + 0 = 7$. Note that a finite edge value specifies the number of finite-valued paths to terminal node 0 by following pointers to the left of the current one in the node; thus, indices are assigned according to lexicographical order. An EV⁺MDD encoding the indices for a set of states can be constructed by first generating an MDD (essentially, an EV⁺MDD where all edge values are either zero or infinity) that represents the set of states, and then filling in the edge values in a bottom-up fashion [25].

4.2. Matrix diagrams

Matrix diagrams [11] combine ideas from both Kronecker representations and decision diagrams. A matrix diagram is a directed, acyclic graph with terminal nodes labeled 0 and 1, and non-terminal nodes with label x_k . A level- k non-terminal node contains an $|\mathcal{S}_k| \times |\mathcal{S}_k|$ matrix of pointers, and each pointer has an associated non-negative real value. A path through the matrix diagram that reaches terminal node 1 corresponds to a non-zero entry in the en-

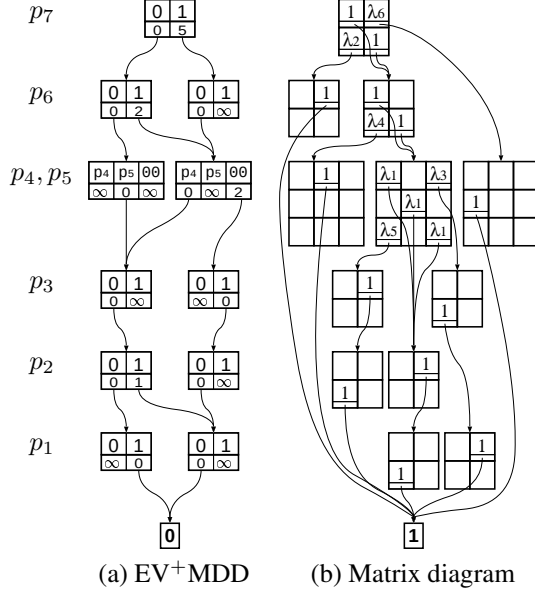


Figure 2. Implicit representations for the running example, with $N = 1$

coded matrix; the value of the entry is determined by multiplying the real values along the path. In a matrix diagram, nodes are said to be *duplicates* if their corresponding pointers and values are equal¹. Like MDDs, matrix diagrams can be reduced by eliminating duplicate nodes. We also eliminate *identity* nodes, which have pointers to the same node and values of one along the diagonal, and pointers to terminal node 0 elsewhere. The overall matrix encoded by the matrix diagram is a square matrix of size $|\hat{S}|$. This is similar to a Kronecker representation [5], which also represents a square matrix $\hat{\mathbf{R}}$ of size $|\hat{S}|$. Indeed, matrix diagrams can be constructed from a Kronecker representation [11]. The transition rate matrix $\mathbf{R}$ is the submatrix corresponding to the “reachable” rows and columns of $\hat{\mathbf{R}}$.

A matrix diagram representing $\hat{\mathbf{R}}$ for the running example is shown in Figure 2(b). Pointers to terminal node 0 and their values are omitted from the figure, for clarity. The rows and columns of the matrices are ordered according to the submarking order shown in the EV⁺MDD nodes of Figure 2(a). The rate of transition from one marking to another is determined by following the appropriate path. For example, according to the matrix diagram, the rate of transition from marking (p_5p_6) to marking $(p_4p_6p_7)$ is found by following the pointer at element $[(\#p_7 = 0), (\#p_7 = 1)]$ of the node for p_7 . Since this pointer skips level p_6 , the node for p_6 must have been an identity node that was elim-

inated; thus, along this path the number of tokens in place p_6 does not change. We then follow the pointer at element $[(p_5), (p_4)]$ of the node for $\{p_4, p_5\}$, which is directly to terminal node 1. Thus, places p_1, p_2 , and p_3 do not change along this path. The product of values along the path, $\lambda_6 \cdot 1$, gives the rate of transition. Note that marking (p_5p_6) is unreachable, yet it still has entries in the matrix $\hat{\mathbf{R}}$.

5. Using implicit structures to compute passage times

The algorithms we use for our implicit representations require that the Petri net model has been decomposed into K subnets. We now describe the constraints that the chosen partition must satisfy.

To construct the reachability set using MDDs, it is necessary to represent the one-step reachability function $\mathcal{N} : \hat{S} \rightarrow 2^{\hat{S}}$, where $\mathcal{N}(\mathbf{i}) = \{\mathbf{j} : \mathbf{i} \rightarrow \mathbf{j}\}$, in a format that can be exploited by MDD-based reachability set generation algorithms. For a Petri net, $\mathcal{N}$ is the union of one-step reachability functions for each transition, $\mathcal{N}_t(\mathbf{i}) = \{\mathbf{j} : \mathbf{i} \xrightarrow{t} \mathbf{j}\}$. Each of these must be expressible as the cross-product of “local” reachability functions: $\mathcal{N}_t = \mathcal{N}_t^K \times \dots \times \mathcal{N}_t^1$ (this is the “product-form” requirement in [8, 26]). For a Petri net model, restrictions occur only when transition guards are not identically one or there are marking-dependent arc cardinalities. Assuming each guard is expressed as a conjunction of terms $g_t(\mathbf{i}) = g_t^1(\mathbf{i}) \wedge \dots \wedge g_t^{n_t}(\mathbf{i})$, then the places contained in each term must belong to the same set $\mathcal{P}_k$. Similarly, if the cardinality of any arc between a place p_i and a transition depends on the number of tokens in place p_j , then p_i and p_j must belong to the same set $\mathcal{P}_k$. The finest partition that satisfies the cross-product requirement can be determined automatically [26].

To construct the reachability set, we must compute $\mathcal{M}_0 \cup \mathcal{N}(\mathcal{M}_0) \cup \mathcal{N}(\mathcal{N}(\mathcal{M}_0)) \cup \dots = \mathcal{N}^*(\mathcal{M}_0)$. We use the saturation algorithm presented in [8] to do so; it uses a small matrix to encode each local reachability function $\mathcal{N}_t^k$, which is possible due to the cross-product requirement.

To efficiently construct the matrix diagram representation for $\hat{\mathbf{R}}$, we require that the contribution due to transition t can be represented as a Kronecker product of matrices: $\hat{\mathbf{R}}_t = \mathbf{W}_t^K \otimes \dots \otimes \mathbf{W}_t^1$. This requires that the rate function for each transition is expressible as the product of local rate functions $\lambda_t(\mathbf{i}) = \lambda_t^K(\mathbf{i}) \cdot \dots \cdot \lambda_t^1(\mathbf{i})$, where $\lambda_t^k(\mathbf{i})$ may depend only on places in $\mathcal{P}_k$. As described in [11], we construct the matrix diagram representation from the Kronecker representation for $\hat{\mathbf{R}}$.

Note that, if we wish to determine the distribution π' for the starting markings by analyzing the embedded DTMC, then we can directly apply the techniques described in [11]: the stationary distribution for the embedded DTMC is deter-

¹To avoid floating-point comparison difficulty, values can be considered equal if they are within some relative tolerance ϵ .

```
BuildTransient(in:  $\mathcal{I}$ ,  $\hat{\mathcal{J}}$ ; out:  $\mathcal{Z}$ )
```

```
1:  $\mathcal{Z} \leftarrow \mathcal{I}$ ;  
2: repeat  
3:    $\mathcal{Y} \leftarrow \mathcal{Z}$ ;  
4:    $\mathcal{Z} \leftarrow \mathcal{Z} \cup (\mathcal{N}(\mathcal{Z}) \setminus \hat{\mathcal{J}})$ ;  
5: until  $\mathcal{Z} = \mathcal{Y}$ 
```

Figure 3. Constructing the MDD for $\mathcal{Z}$

mined by dividing the CTMC distribution by the expected holding times and normalizing the resulting vector. We assume now that the MDD for $\mathcal{S}$ and the matrix diagram for $\hat{\mathbf{R}}$ have been constructed, and the distribution π' has been computed as necessary.

5.1. Generating the absorbing CTMC

We must first determine the reachability set $\mathcal{S}' = \mathcal{Z} \cup \mathcal{A}$ of the absorbing net. Since we already have the MDD representation for $\mathcal{S}$, we can construct the starting and stopping sets using a simple query, as described in [26]: the set of all potential markings satisfying the condition is constructed, and intersected with the set of reachable markings $\mathcal{S}$. Thus, we construct the MDDs for $\hat{\mathcal{I}}$ and $\hat{\mathcal{J}}$, then compute $\mathcal{I} = \hat{\mathcal{I}} \cap \mathcal{S}$ and $\mathcal{A} = \hat{\mathcal{J}} \cap \mathcal{S}$ using MDD operations for set intersection. Sets of markings expressed using computation tree logic (CTL) [14] could also be constructed [13].

The set $\mathcal{Z}$ is constructed by adding markings reachable from $\mathcal{I}$ that have not yet reached a marking in $\hat{\mathcal{J}}$. In terms of set operations, the set $\mathcal{Z}$ can be constructed using the least fixed-point algorithm shown in Figure 3, which adds reachable markings only if they are not contained in the set $\hat{\mathcal{J}}$. The set $\mathcal{Z}$ can also be expressed as the set of states satisfying $E(\hat{\mathcal{S}} \setminus \hat{\mathcal{J}}) \mathcal{S} \mathcal{I}$, where the operator ES is similar to the CTL operator EU , except it operates in the past (i.e., using the inverse of the one-step reachability function) [31]. Informally, this is the set of markings in $\hat{\mathcal{S}} \setminus \hat{\mathcal{J}}$ that have been reached “since” the starting markings $\mathcal{I}$. Note that the inverse of $\mathcal{N}$ can be computed by simply transposing the matrices used to encode the local one-step reachability functions. Thus, given an efficient MDD-based algorithm to compute states satisfying EU (such as [13], which uses saturation), we can construct $\mathcal{Z}$ by either transposing the matrices before invoking the algorithm, or by modifying the algorithm appropriately.

Once we have constructed the MDDs for the sets $\mathcal{Z}$ and $\mathcal{A}$, we can construct the matrix diagram for $\mathbf{R}_{\mathcal{Z}\mathcal{Z}}$ by selecting the rows and columns of $\hat{\mathbf{R}}$ corresponding to $\mathcal{Z}$. This selection operation can be done efficiently when the rows and columns to be selected are represented using MDDs [11]. Similarly, we construct the matrix diagram for $\mathbf{R}_{\mathcal{Z}\mathcal{A}}$ by selecting rows $\mathcal{Z}$ and columns $\mathcal{A}$ from the matrix diagram $\hat{\mathbf{R}}$.

5.2. Numerical solution

Once we have our matrix diagrams for $\mathbf{R}_{\mathcal{Z}\mathcal{Z}}$ and $\mathbf{R}_{\mathcal{Z}\mathcal{A}}$, we are ready to compute the distribution for the uniformized CTMC. To do so, we must be able to compute the probability distribution for the transient states at time n : $\mathbf{p}_{\mathcal{Z}}^n = \pi_{\mathcal{Z}} \cdot \mathbf{P}_{\mathcal{Z}\mathcal{Z}}^n$. We use an explicit vector for $\mathbf{p}_{\mathcal{Z}}^n$, which is computed using $\mathbf{p}_{\mathcal{Z}}^{n+1} = \mathbf{p}_{\mathcal{Z}}^n \cdot \mathbf{P}_{\mathcal{Z}\mathcal{Z}}$. Similarly, we must be able to compute $\Pr\{X = n\} = \mathbf{p}_{\mathcal{Z}}^n \cdot \mathbf{P}_{\mathcal{Z}\mathcal{A}} \cdot \mathbf{1}^T$. Note that $\Pr\{X = n\}$ can, in theory, also be computed using $\Pr\{X > n-1\} - \Pr\{X > n\}$; however, the above vector-matrix product is numerically “safer” (i.e., expected to be more accurate) than floating-point subtraction.

We use an explicit vector $\mathbf{h}_{\mathcal{Z}}$ of expected holding times in each transient state, which can be computed from the row sums of $\mathbf{R}_{\mathcal{Z}\mathcal{Z}}$ and $\mathbf{R}_{\mathcal{Z}\mathcal{A}}$. This allows us to compute $\mathbf{p}_{\mathcal{Z}}^n$ using $\mathbf{R}_{\mathcal{Z}\mathcal{Z}}$ instead of $\mathbf{P}_{\mathcal{Z}\mathcal{Z}}$: after obtaining the vector $\mathbf{p}_{\mathcal{Z}}^n \cdot \mathbf{R}_{\mathcal{Z}\mathcal{Z}}$, we can obtain $\mathbf{p}_{\mathcal{Z}}^{n+1}$ by dividing each element by q (the uniformization constant), and can use $\mathbf{h}_i$ and q to determine the diagonal element $\mathbf{P}_{ii}$. Similarly, we compute $\mathbf{p}_{\mathcal{Z}}^n \cdot \mathbf{P}_{\mathcal{Z}\mathcal{A}} \cdot \mathbf{1}^T$ using $\mathbf{R}_{\mathcal{Z}\mathcal{A}}$ instead and dividing the result by q . Thus, we use uniformization without actually constructing the DTMC; we perform all operations on the CTMC itself and then adjust the results appropriately.

For numerical computation of the PDF or CDF, we must be able to perform the computation $\mathbf{x} \cdot \mathbf{A}$ and the computation $\mathbf{x} \cdot \mathbf{A} \cdot \mathbf{1}^T$, where $\mathbf{A}$ is a matrix diagram and $\mathbf{x}$ is an explicit vector. Since this is the main computational cost of computing the PDF, it is important to use an efficient vector-matrix multiplication algorithm. The algorithm presented in [11] could be used for this purpose; however, it was designed to perform vector-matrix multiplication “by columns”, as required for Gauss-Seidel. This restriction is unnecessary here. We thus develop new vector-matrix multiplication algorithms to improve efficiency.

Our algorithms are based on the block structure captured by the matrix diagram representation: each level- k non-terminal node represents a matrix split into $|S_k|^2$ blocks. Given desired rows and columns $\mathcal{R}$ and $\mathcal{C}$, we can express the vector-matrix product $\mathbf{y} \leftarrow \mathbf{x} \cdot \alpha \cdot \mathbf{A}_{\mathcal{R}\mathcal{C}}$ as

$$[\mathbf{y}_0, \dots, \mathbf{y}_{n-1}] \leftarrow [\mathbf{y}_0, \dots, \mathbf{y}_{n-1}] +$$

$$[\mathbf{x}_0, \dots, \mathbf{x}_{n-1}] \cdot \alpha \cdot \begin{bmatrix} & & & \vdots & \\ \cdots & \alpha_{ij} \cdot \mathbf{A}_{\mathcal{R}_i\mathcal{C}_j} & \cdots & & \\ & & & \vdots & \end{bmatrix}$$

where $\mathcal{R}_i$ is the set of states in $\mathcal{R}$ whose first component is equal to i , and $\mathcal{C}_j$ is similarly defined. Note that if $\mathbf{A}$ is represented using matrix diagrams, then $\mathbf{A}_{\mathcal{R}_i\mathcal{C}_j}$ corresponds to downward pointer $[i, j]$ of the root node, and α_{ij} is the corresponding edge value. Similarly, if $\mathcal{R}$ and $\mathcal{C}$ are represented using EV^+ MDDs (or MDDs), then $\mathcal{R}_i$ is the i^{th} downward pointer in the root node of $\mathcal{R}$, and $\mathcal{C}_j$ is the

<pre> PreMultiply(in: $k, \mathbf{x}, \alpha, \mathbf{A}, \mathcal{R}, \mathcal{C}$; inout: $\mathbf{y}$) 1: • $\mathcal{R}, \mathcal{C}$ are EV^+MDDs for rows, columns; $\mathbf{A}$ is 2: • a matrix diagram. Computes $\mathbf{y} \leftarrow \mathbf{y} + \alpha \mathbf{x} \cdot \mathbf{A}$. 3: if $\mathbf{A} = 1$ then • Terminal node 1, $\mathbf{A}$ is identity 4: $\mathbf{y} \leftarrow \mathbf{y} + \alpha \mathbf{x}$; 5: else if $\text{Top}(\mathbf{A}) < k$ then • Skipped identity node 6: for each $i \in S_k$ do 7: $\mathcal{R}' \leftarrow \mathcal{R}.\text{down}[i]$; $\mathcal{C}' \leftarrow \mathcal{C}.\text{down}[i]$; 8: $\mathbf{x}' \leftarrow \text{Subvector}(\mathbf{x}, \mathcal{R}.\text{value}[i])$; 9: $\mathbf{y}' \leftarrow \text{Subvector}(\mathbf{y}, \mathcal{C}.\text{value}[i])$; 10: PreMultiply($k - 1, \mathbf{x}', \alpha, \mathbf{A}, \mathcal{R}', \mathcal{C}', \mathbf{y}'$); 11: end for 12: else 13: for each $i, j : \mathbf{A}.\text{down}[i, j] \neq 0$ do 14: $\mathcal{R}' \leftarrow \mathcal{R}.\text{down}[i]$; $\mathcal{C}' \leftarrow \mathcal{C}.\text{down}[j]$; 15: $\mathbf{x}' \leftarrow \text{Subvector}(\mathbf{x}, \mathcal{R}.\text{value}[i])$; 16: $\mathbf{y}' \leftarrow \text{Subvector}(\mathbf{y}, \mathcal{C}.\text{value}[j])$; 17: $\alpha' \leftarrow \alpha \cdot \mathbf{A}.\text{value}[i, j]$; $\mathbf{A}' \leftarrow \mathbf{A}.\text{down}[i, j]$; 18: PreMultiply($k - 1, \mathbf{x}', \alpha', \mathbf{A}', \mathcal{R}', \mathcal{C}', \mathbf{y}'$); 19: end for 20: end if </pre>	<pre> PreMultiplySum(in: $k, \mathbf{x}, \alpha, \mathbf{A}, \mathcal{R}$; inout: y) 1: • $\mathcal{R}$ is the set of rows, $\mathbf{A}$ is a matrix diagram. 2: • Computes $y \leftarrow y + \alpha \mathbf{x} \cdot \mathbf{A} \cdot \mathbf{1}^T$. 3: if $\mathbf{A} = 1$ then • Terminal node 1, $\mathbf{A}$ is identity 4: $y \leftarrow y + \alpha \mathbf{x} \cdot \mathbf{1}^T$; 5: else if $\text{Top}(\mathbf{A}) < k$ then • Skipped identity node 6: for each $i \in S_k$ do 7: $\mathcal{R}' \leftarrow \mathcal{R}.\text{down}[i]$; 8: $\mathbf{x}' \leftarrow \text{Subvector}(\mathbf{x}, \mathcal{R}.\text{value}[i])$; 9: PreMultiplySum($k - 1, \mathbf{x}', \alpha, \mathbf{A}, \mathcal{R}', y$); 10: end for 11: else 12: for each i, j such that $\mathbf{A}.\text{down}[i, j] \neq 0$ do 13: $\mathcal{R}' \leftarrow \mathcal{R}.\text{down}[i]$; 14: $\mathbf{x}' \leftarrow \text{Subvector}(\mathbf{x}, \mathcal{R}.\text{value}[i])$; 15: $\alpha' \leftarrow \alpha \cdot \mathbf{A}.\text{value}[i, j]$; $\mathbf{A}' \leftarrow \mathbf{A}.\text{down}[i, j]$; 16: PreMultiplySum($k - 1, \mathbf{x}', \alpha', \mathbf{A}', \mathcal{R}', y$); 17: end for 18: end if </pre>
---	---

Figure 4. Multiplication algorithms for numerical solution

j^{th} downward pointer in the root node of $\mathcal{C}$. Note that the matrix blocks may have different sizes: the dimension of $\mathbf{y}_j$ is equal to $|\mathcal{C}_j|$. For non-empty subsets $\mathcal{C}_j$, note that the starting position of subvector $\mathbf{y}_j$ in vector $\mathbf{y}$ is equal to the number of elements in set $\mathcal{C}$ with first component less than j . If $\mathcal{C}$ is represented using an EV^+ MDD, then this is exactly equal to the edge value associated with downward pointer j in the root node of the EV^+ MDD for $\mathcal{C}$. Thus, we use EV^+ MDD representations for $\mathcal{R}$ and $\mathcal{C}$, and use the edge values to determine the block structure for vectors $\mathbf{x}$ and $\mathbf{y}$. The above multiplication can be performed by visiting each non-zero element of $\mathbf{A}$ and applying the operation $\mathbf{y}_j \leftarrow \mathbf{y}_j + \mathbf{x}_i \cdot \alpha \cdot \alpha_{ij} \cdot \mathbf{A}_{\mathcal{R}_i \mathcal{C}_j}$.

This is the basis for our recursive multiplication algorithm PreMultiply, shown in Figure 4. In the algorithm, down and value refer to the appropriate downward pointer and associated edge value for the given EV^+ MDD or matrix diagram node. Note that the vectors $\mathbf{x}$ and $\mathbf{y}$ in PreMultiply are assumed to be of size $|\mathcal{R}|$ and $|\mathcal{C}|$, respectively. Also, note that the Subvector($\mathbf{x}, i$) operator, which selects the subvector of $\mathbf{x}$ starting at index i , can be implemented efficiently using pointer arithmetic. The algorithm should be invoked at the top level with $k = K$ and $\alpha = 1$.

Note that, unlike the column-based algorithm presented in [11], algorithm PreMultiply does not update elements of vector $\mathbf{y}$ one at a time, and thus is unsuitable for Gauss-Seidel. Algorithm PreMultiply is similar to the “interleaving” vector-matrix multiplication algorithm for Kronecker representations [5]: both algorithms visit each submatrix $\mathbf{A}_{\mathcal{R}_i \mathcal{C}_j}$ only once.

Algorithm PreMultiplySum, also shown in Figure 4, is used to compute $y \leftarrow y + \mathbf{x} \cdot \alpha \cdot \mathbf{A}_{\mathcal{R}\mathcal{C}} \cdot \mathbf{1}^T$. Note that the algorithm is similar to PreMultiply, except uses a scalar y instead of a vector. Assuming the unreachable columns of $\mathbf{A}$ have already been removed, it is unnecessary to traverse the EV^+ MDD for the set $\mathcal{C}$; thus, algorithm PreMultiplySum requires only the EV^+ MDD for the set of rows $\mathcal{R}$ to determine the block sizes for the vector $\mathbf{x}$.

6. Experimental results

We implemented prototypes of both the implicit and explicit solutions in the SMART tool[7]. All results are obtained from 933Mhz Pentium III workstations with 512Mb of memory, running Linux.

We examine several models, starting with the running example. To increase the number of reader and writer processes in the system, we can increase the parameter N ; however, it then becomes necessary to distinguish the writer processes, so that we measure the time from when a *particular* process wants to write until it is able to write. Thus, we must “tag” a particular customer and follow its progress in the system; to do so, we add an extra place for each place in the net that the customer can visit, to track the customer’s position in the queue. This is illustrated in Figure 5(a) for the running example. In the example, place $p_1 \text{ tag}$ represents the position of the tagged token in place p_1 : if the tagged token is not in place p_1 , then $p_1 \text{ tag}$ will contain zero tokens; otherwise, $p_1 \text{ tag}$ will contain a number of tokens equal to one plus the number of tokens ahead of the tagged

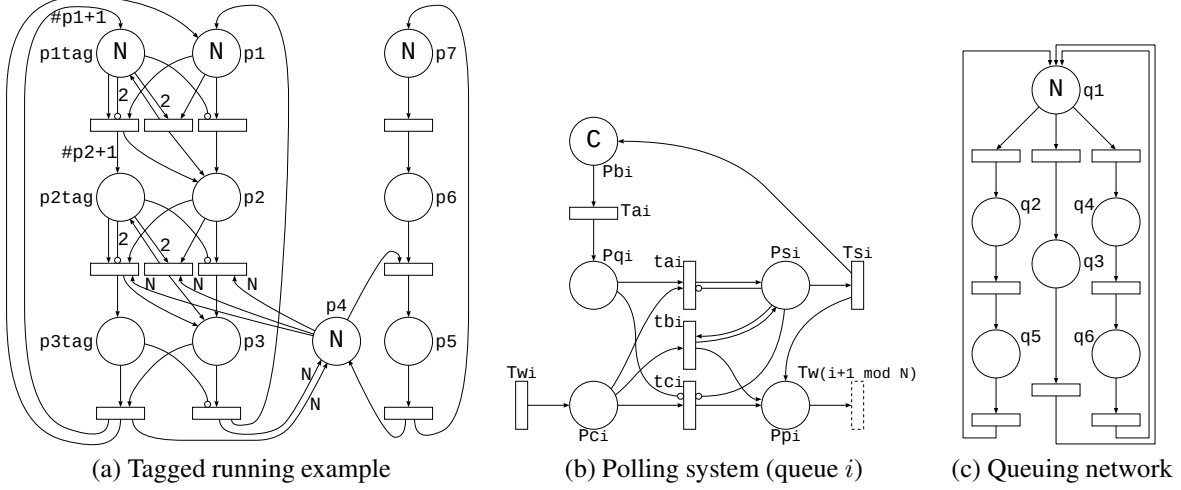


Figure 5. Example models

token in place p_1 . Note that transition t_1 in the original example becomes three distinct transitions: the rightmost one, which handles the case when no tagged token is in place p_1 ; the middle one, which handles the case when a token ahead of the tagged token is consumed; and the leftmost one, which removes the tagged token from p_1 and adds it to p_2 . Note that, in this model, we assume tokens are consumed by transitions in FIFO order.

In the tagged model, the starting set becomes the markings with $\#p_2tag = \#p_2$ (indicating that the process is at the end of the “queue” waiting to write), and the stopping set becomes the markings with a token in $\#p_3tag$ (indicating that the process is writing). For the reported results, we use a two-level partition: $\mathcal{P}_2 = \{p_3, p_3tag, p_4, p_5, p_6, p_7\}$, $\mathcal{P}_1 = \{p_1, p_1tag, p_2, p_2tag\}$.

We also use a multiserver polling system, described in [3], with N queues, $C = 3$ customers per queue, and 3 servers (initialized in place Pp_0). The “untagged” version of a single queue i is shown in Figure 5(b). Note that we use fast timed transitions instead of immediate transitions, and compute the time for a customer to leave queue 0 (the time for a token to go from Pq_0 to Pb_0). Thus, we add “tag” places for Pb_0 , Pq_0 , and Ps_0 , and introduce transitions, similar to the modifications made to the running example. The net is partitioned in a natural way: the places corresponding to a single queue are grouped together. Finally, we use the queuing network described in [18], with N customers, as shown in Figure 5(c). Note that this model requires tagging every place, and adding a sequence bit to count the cycle number of the customer [18]. For this model, we group queues 1 and 3, 2 and 5, and 4 and 6 together to obtain our partition.

For our experiments, we used the steady-state distribution of the embedded DTMC to determine the initial prob-

abilities for the starting markings. Since generation and solution of the embedded DTMC is essentially an orthogonal issue, discussed for example in [5, 11, 19, 20], and since different initial distributions π' could just as easily be used, we do not report the times to compute π' here. We computed the PDF $f(t)$ for 2,000 values of t using (3). Once the PMF for X was known, computing the values for $f(t)$ took only a fraction of a second. Thus, assuming the PMF for X has been saved, additional data points could be computed very quickly if desired.

The computation times required for construction and analysis of the absorbing CTMC (assuming that the initial distribution π' is already known) are shown in Table 1, for an explicit approach using sparse matrix storage, and our implicit approach using EV⁺MDDs and matrix diagrams, using both our new multiplication algorithms and the by-columns algorithm of [11]. The time to build the absorbing CTMC includes generating the sets $\mathcal{A}$ and $\mathcal{Z}$, and generating the matrices $\mathbf{R}_{\mathcal{Z}\mathcal{Z}}$ and $\mathbf{R}_{\mathcal{Z}\mathcal{A}}$. For the implicit approach, we construct $\mathcal{Z}$ using saturation [13]. Note that multiplication by columns requires a second, “upside-down” copy of $\mathcal{Z}$; this is not included in the build time shown in the table. The column “PMF size” indicates the value of n for which $\Pr\{X > n\}$ is smaller than our desired precision $\epsilon = 10^{-6}$, and the columns “Solve”, “PreMultiply”, and “By cols” show the total CPU time required to compute the PMF of X using sparse storage, matrix diagrams with PreMultiply, and matrix diagrams with by-column multiplication, respectively. The “Memory” columns indicate the amount of memory required to represent the matrix of the absorbing CTMC; memory for $\mathcal{Z}$ is not shown. Note that, for numerical solution, all approaches also use a single-precision vector of holding times, a single-precision vector for transient probabilities, and a double-precision vector to compute the

Model	N	$ Z $	PMF size	Sparse			Matrix diagram			
				Memory	Build	Solve	Memory	Build	PreMultiply	By cols.
Read-write	50	1,753,125	614	58.2 Mb	1,017	379	206.0 Kb	3.8	394	491
Read-write	80	11,016,000	615	—	—	—	521.7 Kb	26.8	3,091	3,729
Polling	5	781,536	19,983	39.7 Mb	1,334	8,027	14.0 Kb	0.4	8,653	12,795
Polling	6	5,264,448	21,351	—	—	—	16.1 Kb	3.0	67,546	105,617
QN	18	605,682	825	31.2 Mb	1,146	301	483.2 Kb	75.3	401	504
QN	25	3,562,650	1,014	—	—	—	1.2 Mb	763.7	3,118	3,836

Table 1. CPU times (in seconds) and memory requirements to analyze absorbing CTMCs

next transient probability vector. These vectors are not included in the memory requirements shown in the table.

Note that explicit construction of the absorbing CTMC, while completely general, is significantly more expensive (in terms of both CPU time and memory requirements) than implicit construction, which exploits the Kronecker representation of the underlying CTMC. Note that the matrix diagram representation is much more compact than explicit sparse storage; this savings enables the matrix diagram approach to handle larger solution vectors. Indeed, the dashes in the table correspond to cases where available memory was exceeded by the sparse approach.

In terms of PMF computation time, using sparse storage is the fastest, followed closely by PreMultiply. Indeed, for the polling and readers/writers models, the solution time difference between PreMultiply and sparse storage is within 8% (34% for the queuing network). For the readers/writers and queuing network models, using multiplication by columns is about 25% more expensive than PreMultiply (45% for the polling model), leading to a 30% to 67% overall increase over sparse storage. For the models examined in this paper, when total time required (construction plus solution) is considered, our implicit approach using PreMultiply is the most efficient.

For a rough comparison with other work, for the queuing network model with $N = 18$, our explicit approach required about $19 + 5 = 24$ minutes, while our implicit approach (using PreMultiply) required about $1.3 + 6.7 = 8$ minutes. It was reported in [18] that a distributed Laplace transform inverter obtained the distribution in 26 minutes using 32 workstations with 1.4Ghz processors (it is not specified if this time includes computation of the initial distribution π').

7. Conclusions

We presented a technique to compute response time distributions, expressed as a random variable defined in terms of the time for a stochastic Petri net to evolve from a set of starting markings to a set of stopping markings. When the transitions of the SPN all have exponential firing distributions, the underlying process of the SPN is a CTMC,

and the random variable is a continuous phase-type. Using an explicit approach, it is straightforward to construct and analyze the absorbing CTMC; we also presented an implicit approach, based on MDDs and matrix diagrams. Our algorithm to construct the implicit representation of the absorbing CTMC is partly based on well-established model checking operators. We presented an efficient algorithm for multiplying a vector by a matrix represented as a matrix diagram, which is necessary for numerical solution. The implicit approach can analyze models whose number of states is about an order of magnitude larger; this is especially important since the tagged customer approach increases the size of the underlying Markov chain.

Our approaches are significantly faster than general-purpose Laplace transform inverters, but are currently limited to nets whose transitions have exponentially-distributed firing delays. Directions for future work include the extension of the implicit techniques to handle immediate transitions [2] and transitions with continuous phase-type firing delays [1] (explicit approaches can already handle these cases). This would involve extending the algorithms in [13] to handle immediate events, and developing efficient matrix diagram construction algorithms that support phase-type firing delays.

Acknowledgments

The author thanks Radu Siminiceanu and the SMART developers for help integrating the MDD-based CTL operators (see [13]) with the numerical technique described in this paper.

References

- [1] M. Ajmone Marsan, G. Balbo, A. Bobbio, G. Chiola, G. Conte, and A. Cumani. The effect of execution policies on the semantics and analysis of Stochastic Petri Nets. *IEEE Trans. Softw. Eng.*, 15(7):832–846, July 1989.
- [2] M. Ajmone Marsan, G. Balbo, and G. Conte. A class of Generalized Stochastic Petri Nets for the performance evaluation

of multiprocessor systems. *ACM Transactions on Computer Systems*, 2(2):93–122, May 1984.

- [3] M. Ajmone Marsan, S. Donatelli, F. Neri, and A. Scalia. Approximate GSPN analysis of multiserver polling systems. In *Proceedings of the International Conference on Telecommunication, Distribution and Parallelism (TDP'96)*, La Londe Les Maures, France, June 1996.
- [4] C. Baier, B. Haverkort, H. Hermanns, and J.-P. Katoen. Model checking algorithms for continuous-time Markov chains. *IEEE Trans. Softw. Eng.*, 29(6):524–541, June 2003.
- [5] P. Buchholz, G. Ciardo, S. Donatelli, and P. Kemper. Complexity of memory-efficient Kronecker operations with applications to the solution of Markov models. *INFORMS J. Comp.*, 12(3):203–222, SUMMER 2000.
- [6] G. Chiola. Compiling techniques for the analysis of stochastic Petri nets. *Perf. Eval.*, Sept. 1988.
- [7] G. Ciardo, R. Jones, A. Miner, and R. Siminiceanu. Logical and stochastic modeling with SMART. In *Proc. 13th Int. Conf. on Modelling Techniques and Tools for Computer Performance Evaluation*, 2003. To appear.
- [8] G. Ciardo, G. Luetzgen, and R. Siminiceanu. Saturation: an efficient iteration strategy for symbolic state space generation. In T. Margaria and W. Yi, editors, *Proc. Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, LNCS 2031, pages 328–342, Genova, Italy, Apr. 2001. Springer-Verlag.
- [9] G. Ciardo, R. Marmorstein, and R. Siminiceanu. Saturation unbound. In *Proc. Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, Warsaw, Poland, Apr. 2003. Springer-Verlag. To appear.
- [10] G. Ciardo and A. S. Miner. Storage alternatives for large structured state spaces. In R. Marie, B. Plateau, M. Calzarossa, and G. Rubino, editors, *Proc. 9th Int. Conf. on Modelling Techniques and Tools for Computer Performance Evaluation*, LNCS 1245, pages 44–57, Saint Malo, France, June 1997. Springer-Verlag.
- [11] G. Ciardo and A. S. Miner. A data structure for the efficient Kronecker solution of GSPNs. In P. Buchholz, editor, *8th Int. Workshop on Petri Nets and Performance Models (PNPM'99)*, pages 22–31, Zaragoza, Spain, Sept. 1999. IEEE Comp. Soc. Press.
- [12] G. Ciardo and R. Siminiceanu. Using edge-valued decision diagrams for symbolic generation of shortest paths. In M. D. Aagaard and J. W. O'Leary, editors, *Proc. Fourth International Conference on Formal Methods in Computer-Aided Design (FMCAD)*, LNCS 2517, pages 256–273, Portland, OR, USA, Nov. 2002. Springer-Verlag.
- [13] G. Ciardo and R. Siminiceanu. Structural symbolic CTL model checking of asynchronous systems. In *Proc. 15th Int. Conf. on Computer-Aided Verification (CAV 2003)*, July 2003. To appear.
- [14] E. M. Clarke, O. Grumberg, and D. A. Peled. *Model Checking*. MIT Press, 1999.
- [15] N. J. Dingle, P. G. Harrison, and W. J. Knottenbelt. Response time densities in generalised stochastic Petri net models. In *3rd ACM Workshop on Software and Performance (WOSP 2002)*, pages 46–54, Rome, Italy, July 2002.
- [16] B. L. Fox and P. W. Glynn. Computing Poisson probabilities. *Communications of the ACM*, 31(4):440–445, April 1988.
- [17] D. Gross and D. Miller. The randomization technique as a modeling tool and solution procedure for transient Markov processes. *Operations Research*, 32(2):343–361, 1984.
- [18] P. G. Harrison and W. J. Knottenbelt. Passage time distributions in large Markov chains. In *Proc. 2002 ACM SIGMETRICS Conf. on Measurement and Modeling of Computer Systems*, pages 77–85, Marina Del Rey, CA, June 2002.
- [19] B. Haverkort, A. Bell, and H. Bohnenkamp. On the efficient sequential and distributed generation of very large Markov chains from stochastic Petri nets. In P. Buchholz, editor, *8th Int. Workshop on Petri Nets and Performance Models (PNPM'99)*, pages 12–21, Zaragoza, Spain, Sept. 1999. IEEE Comp. Soc. Press.
- [20] H. Hermanns, J. Kayser, and M. Siegle. Multi terminal binary decision diagrams to represent and analyse continuous time Markov chains. In B. Plateau, W. J. Stewart, and M. Silva, editors, *Numerical Solution of Markov Chains*, pages 188–207, Zaragoza, Spain, June 1999. Prensas Universitarias de Zaragoza.
- [21] T. Kam, T. Villa, R. Brayton, and A. Sangiovanni-Vincentelli. Multi-valued decision diagrams: theory and applications. *Multiple-Valued Logic*, 4(1–2):9–62, 1998.
- [22] F. Kelly and P. Pollet. Sojourn times in closed queueing networks. *Advances in Applied Probability*, 15:638–656, 1983.
- [23] P. Kemper. Transient analysis of superposed GSPNs. *IEEE Trans. Softw. Eng.*, 25(2):182–193, 1999.
- [24] J. McKenna. Asymptotic expansion of the sojourn time distribution functions of jobs in closed, product-form queueing networks. *J. ACM*, 34:985–1103, 1987.
- [25] A. S. Miner. *Data Structures for the Analysis of Large Structured Markov Models*. PhD thesis, The College of William and Mary, Williamsburg, VA, 2000.
- [26] A. S. Miner and G. Ciardo. Efficient reachability set generation and storage using decision diagrams. In H. Kleijn and S. Donatelli, editors, *Application and Theory of Petri Nets 1999 (Proc. 20th Int. Conf. on Applications and Theory of Petri Nets)*, LNCS 1639, pages 6–25, Williamsburg, VA, USA, June 1999. Springer-Verlag.
- [27] M. K. Molloy. Performance analysis using stochastic Petri nets. *IEEE Trans. Comp.*, 31(9):913–917, Sept. 1982.
- [28] J. K. Muppala, K. S. Trivedi, V. Mainkar, and V. G. Kulkarni. Numerical computation of response time distributions using stochastic reward nets. *Annals of Operations Research*, 48:155–184, 1994.
- [29] M. Neuts. *Matrix-Geometric Solutions in Stochastic Models*. The Johns Hopkins University Press, 1981.
- [30] K. Sevcik and I. Mitrani. The distribution of queueing network states at input and output instants. *J. ACM*, 28(2), Apr. 1981.
- [31] F. Somenzi, K. Ravi, and R. Bloem. Analysis of symbolic SCC hull algorithms. In M. Aagaard and J. W. O'Leary, editors, *4th Int. Conf. on Formal Methods in Computer-Aided Design*, LNCS 2517, pages 88–105, Portland, OR, USA, Nov. 2002. Springer-Verlag.